Documento Explicativo — Inciso B: Clasificación de Dígitos con Keras y TensorFlow (Red Densa y CNN Adicional)

Curso / Materia: Redes Neuronales / Internet de las Cosas
Estudiante / Autor: Práctica Académica Reproducible
Entorno de Ejecución: Python 3.10 / TensorFlow 2.21 / Keras 3.12 / NumPy 2.2 / Matplotlib 3.10
Semilla Aleatoria Global: 42

1. Ficha Técnica y Resumen Comparativo de Resultados

Parámetro / Métrica	Red Densa MLP (Requisito Principal)	CNN sobre Dígitos (Experimento Adicional)	Adaline (Inciso A - Referencia)
Tipo de Arquitectura	Perceptrón Multicapa Totalmente Conectado	Convolucional 2D Jerárquico	Neurona Lineal Simple (Sin capas ocultas)
Entrada	Vector 1D: $X \in \mathbb{R}^{400}$	Tensor 2D: $(20, 20, 1)$ en orden Fortran	Vector 1D con sesgo: $X_b \in \mathbb{R}^{401}$
Capas Ocultas	Dense(128, ReLU) $\to$ Dense(64, ReLU)	Conv2D(32) $\to$ Pool $\to$ Conv2D(64) $\to$ Pool $\to$ Dense(64)	Ninguna
Capa de Salida	Dense(10, Softmax)	Dense(10, Softmax)	Salida Lineal + Argmax OvA
Total de Parámetros	**60,234** (235.29 KB)	**121,930** (476.29 KB)	**4,010** (15.66 KB)
Tiempo de Entrenamiento	**11.66 s** (20 épocas)	**22.68 s** (20 épocas)	~3.8 s (50 épocas)
Función de Pérdida	`sparse_categorical_crossentropy`	`sparse_categorical_crossentropy`	Error Cuadrático Medio (ECM)
Optimizador	Adam ($\alpha = 0.001$)	Adam ($\alpha = 0.001$)	Descenso por Gradiente Mini-Batch ($\eta = 0.01$)
Exactitud en Prueba (Test Acc)	**91.60%**	**96.80%**	**85.10%**
Pérdida en Prueba (Test Loss)	**0.3402**	**0.1040**	0.0826 (ECM)
Errores en Prueba (de 1,000)	**84 errores**	**32 errores**	**149 errores**

2. Objetivo de la Práctica (Inciso B)

El objetivo central del Inciso B consiste en superar las limitaciones estructurales de separabilidad lineal observadas en Adaline mediante la implementación de un **Perceptrón Multicapa (MLP)** profundo con TensorFlow y Keras.

Se evalúan sistemáticamente:
1. La reutilización exacta del dataset `NumerosManuscritos.csv` y de los mismos índices de partición 80/20 del Inciso A.
2. El preprocesamiento guiado por el rango numérico observado (evitando normalizaciones erróneas por 255).
3. La derivación metodológica estricta del conjunto de validación **únicamente a partir del conjunto de entrenamiento**, garantizando que el conjunto de prueba quede completamente intacto hasta la evaluación final.
4. El análisis conceptual detallado de la transición entre la regla delta mononivel y el algoritmo de retropropagación del gradiente (*Backpropagation*) con activaciones ReLU y Softmax.
5. La exploración complementaria de una Red Convolucional (CNN) adaptada a la morfología 20×20 en orden Fortran.

---

3. Preprocesamiento Riguroso y Partición Compartida

3.1. Reutilización Determinista de la Partición de A

Para garantizar que las comparaciones de rendimiento entre Adaline y las redes profundas sean formalmente científicas y justas, el script carga los índices guardados en `datos/particion_train_test.npz`:
- **Entrenamiento original (80%):** 4,000 muestras.
- **Prueba reservada (20%):** 1,000 muestras.

3.2. Normalización según Rango Real Observado

En el Inciso A se constató que los píxeles de `NumerosManuscritos.csv` presentan un rango continuo de $[-0.1320, 1.1277]$. Si el alumno dividiera ciegamente entre 255 (como se suele hacer de forma rutinaria con MNIST crudo de enteros $[0, 255]$), las entradas quedarían en una escala microscópica (≈ 0.004), amortiguando los gradientes iniciales e impidiendo que las unidades ReLU alcancen su régimen de activación.

Por ello, se aplicó la transformación min-max basada estrictamente en los extremos observados:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}} = \frac{X - (-0.1320)}{1.1277 - (-0.1320)} \quad \in [0.0, 1.0]$$

3.3. Derivación del Conjunto de Validación sin Fuga de Datos (*No Data Leakage*)

Para evitar cualquier contaminación del conjunto de evaluación final:
- Las 4,000 muestras de entrenamiento se dividieron internamente con semilla fija en:
- **Entrenamiento activo (X_{tr}):** 3,200 imágenes (80% de train).
- **Validación (X_{val}):** 800 imágenes (20% de train).
- El conjunto de prueba (X_{test}, 1,000 muestras) no intervino en ningún cálculo de gradientes, selección de épocas ni ajuste de pesos.

4. Arquitectura de la Red Densa (MLP)

La red neuronal multicapa implementada sigue el diseño solicitado:

```
model_dense = keras.Sequential([
    keras.layers.Input(shape=(400,), name="entrada_400_pixeles"),
    keras.layers.Dense(128, activation='relu', name="capa_oculta_1_128"),
    keras.layers.Dense(64, activation='relu', name="capa_oculta_2_64"),
    keras.layers.Dense(10, activation='softmax', name="capa_salida_10_softmax")
], name="MLP_Digitos_Keras")
```

4.1. Análisis de Capas y Parámetros

1. **Capa Entrada $\to$ Oculta 1:**
2. Conexiones: $400 \times 128 = 51,200$ pesos sinápticos + 128 sesgos = **51,328 parámetros**.
3. Activación: $\text{ReLU}(z) = \max(0, z)$. Introduce no-linealidad, mitigando el problema del desvanecimiento del gradiente (*vanishing gradient*) común en sigmoides.
4. **Capa Oculta 1 $\to$ Oculta 2:**
5. Conexiones: $128 \times 64 = 8,192$ pesos + 64 sesgos = **8,256 parámetros**.
6. Activación: ReLU. Permite componer combinaciones jerárquicas de los rasgos primitivos detectados en la primera capa.
7. **Capa Oculta 2 $\to$ Capa de Salida:**
8. Conexiones: $64 \times 10 = 640$ pesos + 10 sesgos = **650 parámetros**.
9. Activación: $\text{Softmax}(\mathbf{z})*i* = *frac{e^{z_i}}{\sum*$ cuya suma es idénticamente 1.}^9 e^{z_j}}$. Genera un vector de probabilidades continuas $\hat{\mathbf{y}} \in [0, 1]^{10
10. **Total acumulado:** $51,328 + 8,256 + 650 = \mathbf{60,234}$ parámetros entrenables.

---

5. Entrenamiento y Curvas de Aprendizaje (Red Densa)

5.1. Configuración de la Optimización

- **Función de Pérdida:** `sparse_categorical_crossentropy`:
  $$\mathcal{L}*{CE} = -\sum \right)$$}^B \log\left(\hat{y}_{i, y_i
  Penaliza logarítmicamente la probabilidad asignada a la clase verdadera y_i.
- **Optimizador Adam:** Tasa de aprendizaje $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-7}$.
- **Hiperparámetros:** 20 épocas, tamaño de lote 32 (100 pasos de gradiente por época).

5.2. Dinámica de Convergencia

- **Tiempo real medido: 11.66 segundos** en CPU.
- **Evolución de la Pérdida:**
- Época 1: Pérdida train = 1.2401, Pérdida val = 0.5874 (Exactitud val: 85.88%).
- Época 5: Pérdida train = 0.2540, Pérdida val = 0.2838 (Exactitud val: 91.62%).
- Época 10: Pérdida train = 0.1388, Pérdida val = 0.2505 (Exactitud val: 92.12%).
- Época 20: Pérdida train = 0.0491, Pérdida val = 0.2570 (Exactitud val: 93.62%).

5.3. Interpretación de las Curvas (`paso2_curvas_dense.png`)

1. **Fase de Aprendizaje Acelerado (Épocas 1 a 6):** La pérdida de entrenamiento y validación caen drásticamente en paralelo, aumentando la exactitud del 68% a más del 91%.
2. **Estabilización de Validación (Épocas 7 a 20):** La pérdida de validación alcanza un mínimo asintótico en $\approx 0.247 - 0.257$, mientras que la pérdida de entrenamiento continúa descendiendo hacia 0.049. Esto indica el inicio de un sobreajuste incipiente en el conjunto de entrenamiento, pero la exactitud de validación permanece sólida en torno al 93.6%.

---

6. Evaluación en Test y Matriz de Confusión (Red Densa)

6.1. Métricas en Conjunto de Prueba Independiente

Al evaluar el modelo final sobre las 1,000 imágenes de prueba:
- **Pérdida en Test:** 0.3402
- **Exactitud en Test: 91.60%** (848 aciertos de 1,000 muestras).
- **Total de errores: 84 errores** (frente a los 149 errores de Adaline).
- **Reducción de errores:** $\frac{149 - 84}{149} = \mathbf{43.6\%}$ de reducción en la tasa de error absoluto.

6.2. Análisis de la Matriz de Confusión (`paso3_matriz_confusion_dense.png`)

Al contrastar la matriz de confusión de la red densa con la de Adaline:
1. **Dígito 5:** En Adaline sufrió 11 confusiones severas con el 8. En la red densa, la capacidad no lineal de ReLU para detectar la esquina superior abierta del 5 redujo las confusiones con el 8 a solo 4 casos.
2. **Dígito 0 y Dígito 1:** Mantienen precisiones sobresalientes superiores al 97%.
3. **Dígito 9 y Dígito 7:** Continúan siendo el par con mayor correlación residual (6 casos de confusión), debido a que comparten el trazo horizontal superior y la diagonal descendente.

6.3. Galería de Predicciones (`paso4_ejemplos_predicciones_dense.png`)

- **Aciertos con alta confianza:** Dígitos claramente rotulados con certidumbres superiores al 98.5% (por ejemplo, dígitos 9, 1, 2 y 8 con probabilidades de Softmax $P \approx 1.0$).
- **Casos de error representativos:**
- Un dígito 3 escrito de forma sumamente angulosa fue predicho como 2 ($P = 40.1\%$).

- Un dígito 4 con la barra vertical extendida y sin lazo cerrado fue predicho como 9 ($P = 90.0\%$).
- Un dígito 2 con bucle inferior muy denso fue interpretado como 8 ($P = 95.9\%$).

---

7. Experimento Adicional: Red Convolucional (CNN) sobre Dígitos

(Requisito 7: Marcado explícitamente como experimento adicional, no sustituto del requisito principal)

7.1. Reconstrucción Tensorial 2D

A diferencia de la red densa que procesa vectores planos unidimensionales de 400 píxeles, la convolución explota la correlación espacial bidimensional local. Para ello, cada muestra fue reconstruida con la orientación original de MATLAB:

```
def a_tensor_cnn(X_mat):
    return np.array([v.reshape(20, 20, 1, order='F') for v in X_mat], dtype=np.float32)
```

7.2. Arquitectura Convolucional

- `Input(shape=(20, 20, 1))`
- `Conv2D(32, kernel=(3,3), padding='same', activation='relu')` $\to$ mapa de $(20, 20, 32)$
- `MaxPooling2D((2, 2))` $\to$ reducción espacial a $(10, 10, 32)$
- `Conv2D(64, kernel=(3,3), padding='same', activation='relu')` $\to$ mapa de $(10, 10, 64)$
- `MaxPooling2D((2, 2))` $\to$ reducción espacial a $(5, 5, 64)$
- `Flatten()` $\to$ vector de $5 \times 5 \times 64 = 1,600$ rasgos
- `Dense(64, activation='relu')`
- `Dense(10, activation='softmax')`
- **Total de Parámetros:** 121,930 (476.29 KB).

7.3. Desempeño Sobresaliente de la CNN

- **Tiempo de entrenamiento: 22.68 segundos** en CPU.
- **Pérdida en Test: 0.1040** (tres veces menor que la red densa).
- **Exactitud en Test: 96.80%** (¡solo 32 errores en todo el conjunto de prueba de 1,000 imágenes!).

---

8. Diferencias Conceptuales: Adaline vs Red Multicapa con Backpropagation

Dimensión Conceptual	Adaline (ADAptive LInear NEuron)	Red Multicapa Profunda (MLP con Keras)
Topología y Profundidad	Capa única de entrada a salida. Sin capas intermedias ni espacio latente.	Múltiples capas ocultas jerárquicas intermedias.
Frontera de Separabilidad	Hiperplano estrictamente lineal en $\mathbb{R}^{400}$. Incapaz de resolver problemas no linealmente separables (análogo al XOR generalizado).	Superficies hiperdimensionales no lineales altamente complejas y flexibles (aproximador universal de funciones).
Funciones de Activación	Función Identidad (Salida Lineal continua $z = \mathbf{w}^T \mathbf{x} + b$). La discretización sólo ocurre al final.	Funciones no lineales continuas y diferenciables por partes en capas ocultas (ReLU) y Softmax probabilístico en salida.
Objetivo Matemático	Mínimos cuadrados sobre objetivos bipolares $t \in {-1, +1}$ mediante Error Cuadrático Medio (ECM).	Maximización de la verosimilitud de clases mediante entropía cruzada multiclase (*Categorical Cross-Entropy*).
Algoritmo de Aprendizaje	**Regla Delta (Widrow-Hoff / LMS):** Actualización directa y mononivel: $\Delta \mathbf{w} = \eta e \mathbf{x}$. No requiere propagación encadenada.	**Backpropagation:** Aplicación sistemática de la regla de la cadena multivariable para computar $\frac{\partial \mathcal{L}}{\partial W^{[l]}}$, retropropagando tensores de error capa por capa.
Extracción de Rasgos	Correlación espacial rígida con una plantilla fija de píxeles (campo receptivo estático).	Representación jerárquica de características: las primeras capas extraen bordes locales; las capas profundas sintetizan conceptos abstractos.

9. Conclusiones Académicas del Inciso B

1. **Salto Cuantitativo y Cualitativo:**
   Al pasar de un modelo lineal (Adaline: 85.10%) a una red densa multicapa con activaciones no lineales (MLP: 91.60%), la exactitud mejoró en **+6.5 puntos porcentuales**, recortando los fallos de 149 a 84.
2. **Capacidad de Desambiguación:**
   La no linealidad de ReLU demostró ser determinante para distinguir caracteres con solapamientos intensos de trazos (especialmente el par 5 y 8).
3. **El Valor de la Topología Espacial (CNN):**
   El experimento adicional con CNN demostró que, al incorporar filtros convolucionales locales e invarianza traslacional, la exactitud se disparó al **96.80%**, con tan solo 32 fallos en prueba.
4. **Preparación para el Inciso C:**
   Estos hallazgos sientan las bases teóricas y empíricas para el **Inciso C**, donde contrastaremos formalmente una arquitectura Densa frente a una Convolucional sobre un dataset de mayor complejidad visual: **Fashion-MNIST** (60,000 imágenes con texturas, pliegues y variaciones de siluetas de ropa y calzado).